

# UART串行总线舵机SDK使用手册(C++)

## 舵机对象的创建与初始化

导入依赖，

```
// 标准库
#include <iostream>

// 导入串口通信库的头文件
#include "CSerialPort/SerialPort.h"
// 导入串口总线舵机库的头文件
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"
```

`FashionStar_UartServoProtocol` 用来处理舵机的底层通信协议的逻辑（数据帧的收发，数据校验等）。

`FashionStar_UartServo` 是舵机的SDK，是在协议上层的更高一级的封装。

使用命名空间，

```
using namespace std;
using namespace fsuservo;
```

配置串口总线舵机转接板对应的端口号。

```
// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
```

创建一个串口总线舵机通信协议对象 `FSUS_Protocol`，默认波特率为 115200。

```
// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
```

创建一个 `FSUS_servo` 舵机对象，创建的时候需要传入舵机的ID，以及通信协议对象的指针 `&protocol`。

舵机的ID取值范围为 0-254

```
#define SERVO_ID 0 //舵机ID号
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);
```

## 舵机通讯检测

### API-ping

调用舵机的 ping() 函数用于舵机的通信检测，判断舵机是否在线。

```
bool isOnline = uservo.ping(); // 舵机通讯检测
```

## 例程源码

servo\_ping.cpp

```
/*
 * 舵机通讯检测
 * -----
 * 作者：阿凯|kyle
 * 邮箱：kyle.xing@fashionstar.com.hk
 * 更新时间：2021/06/12
 */

#include <iostream>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

int main(){
    cout << "Example Uart Servo Ping" << endl;
```

```

while(true){
    // 通信检测
    bool is_online = servo0.ping();
    // 打印日志
    cout << "Servo ID = " << SERVO_ID << " , is ";
    if (is_online){
        cout << "online" << endl;
    }else{
        cout << "offline" << endl;
    }
    // 延时1s
    protocol.delay_ms(1000);
}
}

```

## 舵机阻尼模式

### API- setDamping

设置舵机为阻尼模式。

```
void FSUS_Servo::setDamping(FSUS_POWER_T power)
```

#### 输入参数

- `power` 舵机的功率，单位为mW. 功率值越大，旋转舵机的时候阻尼力也就越大。

#### 使用示例

```

#define DAMPING_POWER 800 // 阻尼模式下的功率(单位mw) 500,800,1000

uservo.setDamping(DAMPING_POWER);

```

## 例程源码

servo\_damping.cpp

```

/*
 * 设置舵机为阻尼模式
 * 调整参数`DAMPING_POWER`感受不同的阻尼力
 * -----
 * 作者：阿凯|kyle
 * 邮箱：kyle.xing@fashionstar.com.hk
 * 更新时间：2021/06/12
 */
#include <iostream>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

```

```

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
#define DAMPING_POWER 800 // 阻尼模式的功率

using namespace std;
using namespace fsuser;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

int main(){
    // 打印例程信息
    cout << "Set Servo Mode To Damping" << endl;
    // 设置舵机为阻尼模式
    servo0.setDamping(DAMPING_POWER);
    cout << "Set Damping Power = " << DAMPING_POWER << endl;
}

```

## 舵机角度查询

### API-queryAngle

查询舵机当前的真实角度，向舵机发送角度查询指令，并将角度值赋值给舵机对象的 curAngle 属性。

```
FSUS_SERVO_ANGLE_T FSUS_Servo::queryRawAngle()
```

#### 输入参数

- <无>

#### 输出参数

- curRawAngle 舵机当前的真实角度

#### 使用示例

##### 示例1

```
float curAngle = uservo.queryRawAngle()
```

##### 示例2

```
// 舵机角度查询（更新角度）
uservo.queryRawAngle();
// 通过.curRawAngle访问当前的真实角度
uservo.curRawAngle
```

## 例程源码-查询角度(单圈)

servo\_query\_angle.cpp

```
/*
 * 舵机角度回读实验
 * 用手掰动舵机，角度回读并将角度读数通过SPI发送
 * -----
 * 作者：阿凯|kyle
 * 邮箱：kyle.xing@fashionstar.com.hk
 * 更新时间：2021/06/12
 */
#include <iostream>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
#define DAMPING_POWER 800 // 阻尼模式的功率

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

int main(){
    // 打印例程信息
    cout << "Query Servo Angle" << endl;
    // 设置舵机为阻尼模式
    servo0.setDamping(DAMPING_POWER);

    while(true){
        // 查询(更新)舵机角度
        servo0.queryRawAngle();
        // 输出查询信息
        cout << "Servo Angle: " << servo0.curRawAngle << endl;
        // 延时1s
        protocol.delay_ms(1000);
    }
}
```

```
}  
  
}
```

## 舵机轮式模式

### API-wheelStop

轮式模式，停止旋转。

#### 函数原型

```
void FSUS_Servo::wheelStop()
```

#### 输入参数

<无>

### API-wheelRun

轮子持续旋转。

#### 函数原型

```
void FSUS_Servo::wheelRun(uint8_t is_cw)
```

#### 输入参数

- `is_cw`：轮子的旋转方向
  - 0：逆时针
  - 1：顺时针

### API-wheelRunNTime

轮子旋转特定的时间。

#### 函数原型

```
void FSUS_Servo::wheelRunNTime(uint8_t is_cw, uint16_t time_ms)
```

#### 输入参数

- `is_cw`：轮子的旋转方向
  - 0：逆时针
  - 1：顺时针
- `time_ms`：持续旋转的时间，单位为ms

# API-wheelRunNCircle

轮子旋转特定的圈数。

## 函数原型

```
void FSUS_Servo::wheelRunNCircle(uint8_t is_cw, uint16_t circle_num)
```

## 输入参数

- `is_cw`：轮子的旋转方向
  - 0：逆时针
  - 1：顺时针
- `circle_num`：轮子旋转的圈数

## 例程源码

servo\_wheel\_mode.cpp

```
/*
 * 测试舵机轮式模式
 * 提示：拓展板上电之后，记得按下Arduino的RESET按键
 * -----
 * 作者：阿凯|kyle
 * 邮箱：kyle.xing@fashionstar.com.hk
 * 更新时间：2021/06/12
 */

#include <iostream>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

/* 轮子持续旋转指令与停止指令测试 */
void testWheelRunAndStop(){
    servo0.wheelRun(FSUS_CCW); // 轮子持续旋转，方向为逆时针
```

```

    protocol.delay_ms(2000);          // 等待2s
    servo0.wheelStop();
    protocol.delay_ms(2000);          // 等待2s
    servo0.wheelRun(FSUS_CW); // 轮子持续旋转
    protocol.delay_ms(2000);          // 等待2s
    servo0.wheelStop();
    protocol.delay_ms(2000);          // 等待2s
}

/* 测试轮子旋转特定的时间 */
void testWheelRunNTime(){
    servo0.wheelRunNTime(FSUS_CW, 5000); // 轮子持续旋转5s(顺时针)
    protocol.delay_ms(5000);
    servo0.wheelRunNTime(FSUS_CCW, 5000); // 轮子持续旋转5s(逆时针)
    protocol.delay_ms(5000);
}

/* 测试轮子旋转特定的圈数 */
void testWheelRunNCircle(){
    uint16_t nCircle = 2; // 旋转圈数
    uint16_t delayMsEstimate = (uint16_t)(360.0 * nCircle / servo0.speed *
1000); // 估计旋转的时间
    servo0.wheelRunNCircle(FSUS_CW, 2); // 轮子持续旋转2圈(顺时针)
    protocol.delay_ms(delayMsEstimate);          // 等到轮子旋转到特定的位置

    servo0.wheelRunNCircle(FSUS_CCW, 2); // 轮子持续旋转2圈(逆时针)
    protocol.delay_ms(delayMsEstimate);          // 等到轮子旋转到特定的位置}
}

int main(){
    // 打印例程信息
    cout << "Test Wait" << endl;

    // 测试持续旋转与停止
    // testRunAndStop();

    // 测试旋转特定的时间
    // testWheelRunNTime();

    // 测试旋转特定的圈数
    testWheelRunNCircle();
}

```

## 设置舵机角度

### API- setAngle

设定舵机的角度。

**函数原型**



```
/* 设置舵机的原始角度 */  
void FSUS_Servo::setRawAngle(FSUS_SERVO_ANGLE_T rawAngle, FSUS_INTERVAL_T  
interval, FSUS_POWER_T power)
```

```
/* 设置舵机的原始角度 */  
void FSUS_Servo::setRawAngle(FSUS_SERVO_ANGLE_T rawAngle, FSUS_INTERVAL_T  
interval)
```

```
/* 设置舵机的原始角度 */  
void FSUS_Servo::setRawAngle(FSUS_SERVO_ANGLE_T rawAngle)
```

### 输入参数

- `rawAngle`：舵机的目标角度，单位°
- `interval`：舵机旋转的周期，单位ms
- `power`：最大功率，单位mW

## API-`setRawAngleByInterval`

### 函数原型

```
// 设置舵机的原始角度(指定周期)  
void FSUS_Servo::setRawAngleByInterval(FSUS_SERVO_ANGLE_T rawAngle,  
FSUS_INTERVAL_T interval, FSUS_INTERVAL_T t_acc, FSUS_INTERVAL_T t_dec,  
FSUS_POWER_T power)
```

### 输入参数

- `rawAngle`：舵机的目标角度，单位°
- `interval`：舵机旋转的周期，单位ms
- `t_acc`：加速时间
- `t_dec`：减速时间
- `power`：最大功率，单位mW

## API-`setRawAngleByVelocity`

### 函数原型

```
// 设定舵机的原始角度(指定转速)  
void FSUS_Servo::setRawAngleByVelocity(FSUS_SERVO_ANGLE_T rawAngle,  
FSUS_SERVO_SPEED_T velocity, FSUS_INTERVAL_T t_acc, FSUS_INTERVAL_T t_dec,  
FSUS_POWER_T power)
```

### 输入参数

- `rawAngle`：舵机的目标角度，单位°
- `velocity`：舵机旋转的转速，单位°/s
- `t_acc`：加速时间

- `t_dec`: 减速时间
- `power`: 最大功率, 单位mW

## API- `isStop`

判断舵机是否在旋转，是否是静止。该函数在执行的时候，会先查询舵机当前的角度，返回对比跟目标角度 `targetAngle` 之间的差值是否小于控制死区。

### 函数原型

```
bool FSUS_Servo::isStop()
```

### 输入参数

<无>

### 返回参数

- `is_stop`: 是否停止
  - `true`: 舵机已经到达目标角度，已停止旋转
  - `false`: 舵机还没有到达目标角度，正在旋转

## API- `setRange`

设置舵机的角度范围。

### 函数原型

```
void FSUS_Servo::setAngleRange(FSUS_SERVO_ANGLE_T minAngle, FSUS_SERVO_ANGLE_T maxAngle)
```

### 输入参数

- `minAngle`: 舵机角度下限
- `maxAngle`: 舵机角度上限

### 输出参数

<无>

## 例程源码

```
/*
 * 设置舵机的角度(单圈模式)
 * 提示：拓展板上电之后，记得按下Arduino的RESET按键
 * -----
 * 作者：阿凯|kyle
 * 邮箱：kyle.xing@fashionstar.com.hk
 * 更新时间：2021/06/12
 */
#include <iostream>
```

```

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
#define DAMPING_POWER 800 // 阻尼模式的功率

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

// 轨迹规划参数定义
uint16_t interval; // 运行周期 单位ms
uint16_t t_acc; // 加速时间 单位ms
uint16_t t_dec; // 减速时间 单位ms
float velocity; // 目标转速 单位°/s

/* 等待并报告当前的角度*/
void waitAndReport(FSUS_Servo* servo){
    servo->wait(); // 等待舵机旋转到目标角度
    cout << "Real Angle = " << servo->curRawAngle << ", Target Angle = " <<
servo->targetRawAngle << endl;
    protocol.delay_ms(2000); // 暂停2s
}

int main(){
    // 打印例程信息
    cout << "Set Servo Angle" << endl;

    cout << "Set Angle = 90 deg" << endl;
    servo0.setRawAngle(90.0); // 设置舵机的角度
    waitAndReport(&servo0);

    cout << "Set Angle = -90 deg" << endl;
    servo0.setRawAngle(-90);
    waitAndReport(&servo0);

    cout << "Set Angle = 90 deg - Set Interval = 500ms" << endl;
    interval = 1000;
    t_acc = 100;
    t_dec = 100;
    servo0.setRawAngleByInterval(90, interval, t_acc, t_dec, 0);
    waitAndReport(&servo0);

    cout << "Set Angle = -90 deg - Set Velocity = 200 dps" << endl;
    velocity = 200.0;

```

```
t_acc = 100;
t_dec = 100;
servo0.setRawAngleByVelocity(-90, velocity, t_acc, t_dec, 0);
waitAndReport(&servo0);

}
```

## 舵机阻塞式等待

### API-wait

等待舵机旋转到目标角度，阻塞式。

#### 函数原型

```
void FSUS_Servo::wait()
```

#### 输入参数

<无>

#### 输出参数

<无>

## 例程源码

servo\_wait.cpp

```
/*
 * 测试wait()函数, 轮询角度直到舵机旋转到目标位置
 * -----
 * 作者: 阿凯 | kyle
 * 邮箱: kyle.xing@fashionstar.com.hk
 * 更新时间: 2021/06/12
 */

#include <iostream>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
```

```

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

int main(){
    // 打印例程信息
    cout << "Test Wait" << endl;

    cout << "Set Angle = 90.0" << endl;
    servo0.setRawAngle(90.0); // 设置舵机的角度
    servo0.wait();
    protocol.delay_ms(500);
    servo0.queryRawAngle();
    cout << "Real Angle = " << servo0.curRawAngle << endl;

    protocol.delay_ms(2000);

    cout << "Set Angle = -90.0" << endl;
    servo0.setRawAngle(-90.0); // 设置舵机的角度
    servo0.wait();
    protocol.delay_ms(500);
    servo0.queryRawAngle();
    cout << "Real Angle = " << servo0.curRawAngle << endl;

    protocol.delay_ms(2000);
}

```

## 设置舵机角度-多圈模式

### API-`setRawAngleMTurn`

#### 函数原型

```

// 设定舵机的原始角度(多圈)
void FSUS_Servo::setRawAngleMTurn(FSUS_SERVO_ANGLE_T rawAngle,
FSUS_INTERVAL_T_MTURN interval, FSUS_POWER_T power)

```

```

// 设定舵机的原始角度(多圈)
void FSUS_Servo::setRawAngleMTurn(FSUS_SERVO_ANGLE_T rawAngle,
FSUS_INTERVAL_T_MTURN interval)

```

```

// 设定舵机的原始角度(多圈)
void FSUS_Servo::setRawAngleMTurn(FSUS_SERVO_ANGLE_T rawAngle)

```

#### 输入参数

- `rawAngle`: 舵机的目标角度, 单位°
- `interval`: 舵机旋转的周期, 单位ms

- `power`：最大功率，单位mW

#### 输出参数

<无>

## API-`setRawAngleByInterval`

#### 函数原型

```
// 设定舵机的原始角度(多圈+指定周期)
void FSUS_Servo::setRawAngleMTurnByInterval(FSUS_SERVO_ANGLE_T rawAngle,
FSUS_INTERVAL_T_MTURN interval, FSUS_INTERVAL_T t_acc, FSUS_INTERVAL_T t_dec,
FSUS_POWER_T power)
```

#### 输入参数

- `rawAngle`：舵机的目标角度，单位°
- `interval`：舵机旋转的周期，单位ms
- `t_acc`：加速时间，单位ms
- `t_dec`：减速时间，单位ms
- `power`：最大功率，单位mW

#### 输出参数

<无>

## API-`setRawAngleMTurnByVelocity`

#### 函数原型

```
// 设定舵机的原始角度(多圈+指定转速)
void FSUS_Servo::setRawAngleMTurnByVelocity(FSUS_SERVO_ANGLE_T rawAngle,
FSUS_SERVO_SPEED_T velocity, FSUS_INTERVAL_T t_acc, FSUS_INTERVAL_T t_dec,
FSUS_POWER_T power)
```

#### 输入参数

- `rawAngle`：舵机的目标角度，单位°
- `velocity`：舵机旋转的速度，单位°/s
- `t_acc`：加速时间，单位ms
- `t_dec`：减速时间，单位ms
- `power`：最大功率，单位mW

#### 输出参数

<无>

# 例程源码

servo\_set\_angle\_mturn.cpp

```
/*
 * 设置舵机的角度(多圈模式)
 * -----
 * 作者: 阿凯|kyle
 * 邮箱: kyle.xing@fashionstar.com.hk
 * 更新时间: 2021/06/12
 */

#include <iostream>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
#define DAMPING_POWER 800 // 阻尼模式的功率

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

// 轨迹规划参数定义
uint16_t interval; // 运行周期 单位ms
uint16_t t_acc; // 加速时间 单位ms
uint16_t t_dec; // 减速时间 单位ms
float velocity; // 目标转速 单位°/s

/* 等待并报告当前的角度*/
void waitAndReport(FSUS_Servo* servo){
    servo->wait(); // 等待舵机旋转到目标角度
    cout << "Real Angle = " << servo->curRawAngle << ", Target Angle = " <<
servo->targetRawAngle << endl;
    protocol.delay_ms(2000); // 暂停2s
}

int main(){
    // 打印例程信息
    cout << "Set Servo Angle" << endl;

    cout << "Set Angle = 900 deg" << endl;
    servo0.setRawAngleMTurn(900.0); // 设置舵机的角度
    waitAndReport(&servo0);
}
```

```

cout << "Set Angle = -900.0 deg" << endl;
servo0.setRawAngleMTurn(-900.0);
waitAndReport(&servo0);

cout << "Set Angle = 900 deg - Set Interval = 10s" << endl;
interval = 10000;
t_acc = 100;
t_dec = 100;
servo0.setRawAngleMTurnByInterval(900, interval, t_acc, t_dec, 0);
waitAndReport(&servo0);

cout << "Set Angle = -900 deg - Set Velocity = 200 dps" << endl;
velocity = 200.0;
t_acc = 100;
t_dec = 100;
servo0.setRawAngleMTurnByVelocity(-900, velocity, t_acc, t_dec, 0);
waitAndReport(&servo0);

}

```

## 输出日志

```

Set Angle = 900°
Set Servo Angle
Set Angle = 900°
Real Angle = 899.0 Target Angle = 900.0
Set Angle = -900.0°
Real Angle = -899.0 Target Angle = -900.0
Set Angle = 900° - Set Interval = 10s
Real Angle = 899.0 Target Angle = 900.0
Set Angle = -900° - Set Velocity = 200°/s
Real Angle = -899.0 Target Angle = -900.0

```

# 舵机扭力开关

## API- setTorque

### 函数原型

```
void FSUS_Servo::setTorque(bool enable)
```

### 输入参数

- enable: 扭力是否开启
  - true: 开启扭力
  - false: 关闭扭力

### 使用示例

```
uservo.setTorque(true); // 开启扭力
```



## 例程源码

servo\_torque.cpp

```
/*
 * 测试舵机扭力开关
 * -----
 * 作者：阿凯|kyle
 * 邮箱：kyle.xing@fashionstar.com.hk
 * 更新时间：2021/06/12
 */

#include <iostream>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

int main(){
    // 打印例程信息
    cout << "Set Torque" << endl;
    // 开启扭力
    servo0.setTorque(true);
    // 关闭扭力
}
```

## 舵机标定

### API-calibration

在 `FSUS_Servo` 类里面, 有两个跟标定相关的参数:

```
class FSUS_Servo{
public:
    ...
    float kAngleReal2Raw; // 舵机标定数据-舵机角度与位置之间的比例系数
    float bAngleReal2Raw; // 舵机标定数据-舵机角度与位置转换过程中的偏移量
    ...
}
```

舵机真实角度跟原始角度的映射关系如下：

$$angleRaw = kAngleReal2Raw \cdot angleReal + bAngleReal2Raw$$

## 函数原型

```
void FSUS_Servo::calibration(FSUS_SERVO_ANGLE_T rawA, FSUS_SERVO_ANGLE_T realA,
    FSUS_SERVO_ANGLE_T rawB, FSUS_SERVO_ANGLE_T realB)
```

## 输入参数

- rawA：在位置A时刻舵机原始的角度
- realA：在位置A时刻舵机真实的角度
- rawB：在位置B时刻舵机原始的角度
- realB：在位置B时刻舵机真实的角度

## 使用示例

```
// 设置舵机的标定点
// 样本1
#define SERVO_REAL_ANGLE_A 90 // 舵机真实角度
#define SERVO_RAW_ANGLE_A -86.2 // 舵机原始角度
// 样本2
#define SERVO_REAL_ANGLE_B -90 // 舵机真实角度
#define SERVO_RAW_ANGLE_B 91.9 // 舵机原始角度

// 输入舵机标定数据
uservo.calibration(
    SERVO_RAW_ANGLE_A, SERVO_REAL_ANGLE_A, \
    SERVO_RAW_ANGLE_B, SERVO_REAL_ANGLE_B);
```

## 函数原型

```
void FSUS_Servo::calibration(float kAngleReal2Raw, float bAngleReal2Raw);
```

## 输入参数

- kAngleReal2Raw：舵机标定数据-舵机角度与位置之间的比例系数
- bAngleReal2Raw：舵机标定数据-舵机角度与位置转换过程中的偏移量

## API- angleReal2Raw

舵机真实角度转换为舵机原始角度

### 函数原型

```
// 真实角度转化为原始角度
FSUS_SERVO_ANGLE_T FSUS_Servo::angleReal2Raw(FSUS_SERVO_ANGLE_T realAngle);
```

### 输入参数

- `realAngle`: 舵机真实角度

### 返回参数

- `rawAngle`: 舵机原始角度

## API- angleRaw2Real

舵机原始角度转化为真实角度

### 函数原型

```
// 原始角度转换为真实角度
FSUS_SERVO_ANGLE_T FSUS_Servo::angleRaw2Real(FSUS_SERVO_ANGLE_T rawAngle);
```

### 输入参数

- `rawAngle`: 舵机原始角度

### 返回参数

- `realAngle`: 舵机真实角度

## 例程源码

```
/*
 * -----
 * 作者: 阿凯|kyle
 * 邮箱: kyle.xing@fashionstar.com.hk
 * 更新时间: 2021/06/12
 */
#include <iostream>
#include <string>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号
```

```

// 设置舵机的标定点
// 样本1
#define SERVO_REAL_ANGLE_A 90 // 舵机真实角度
#define SERVO_RAW_ANGLE_A -86.2 // 舵机原始角度
// 样本2
#define SERVO_REAL_ANGLE_B -90 // 舵机真实角度
#define SERVO_RAW_ANGLE_B 91.9 // 舵机原始角度

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

int main(){
    // 输入舵机标定数据
    servo0.calibration(
        SERVO_RAW_ANGLE_A, SERVO_REAL_ANGLE_A, \
        SERVO_RAW_ANGLE_B, SERVO_REAL_ANGLE_B);

    // 打印舵机标定数据
    cout << "kAngleReal2Raw = " << servo0.kAngleReal2Raw << "; bAngleReal2Raw = " << servo0.bAngleReal2Raw << endl;

    while(true){
        // 设置舵机角度
        cout << "Set Servo Angle = 90 deg" << endl;
        servo0.setAngle(90.0);
        servo0.wait();

        // 延时2s
        protocol.delay_ms(2000);

        cout << "Set Servo Angle = -90 deg" << endl;
        servo0.setAngle(-90.0);
        servo0.wait();

        // 延时2s
        protocol.delay_ms(2000);
    }
}

```

## 舵机转速设置

# API- setSpeed

## 函数原型

```
void FSUS_Servo::setSpeed(FSUS_SERVO_SPEED_T speed)
```

## 输入参数

- `speed` 舵机的平均转速，单位°/s

## 返回参数

<无>

## 舵机数据读取

## 示例源码

servo\_data\_read.cpp

```
/*
 * -----
 * 作者：阿凯 | kyle
 * 邮箱：kyle.xing@fashionstar.com.hk
 * 更新时间：2021/06/12
 */
#include <iostream>
#include <string>

#include "CSerialPort/SerialPort.h"
#include "FashionStar/UServo/FashionStar_UartServoProtocol.h"
#include "FashionStar/UServo/FashionStar_UartServo.h"

// 串口总线舵机配置
// 设置串口总线Servo转接板的端口号
#ifdef _WIN32
#define SERVO_PORT_NAME "COM8" // windows下端口号名称 COM{}
#else
#define SERVO_PORT_NAME "/dev/ttyUSB0" // Linux下端口号名称 /dev/ttyUSB{}
#endif
#define SERVO_ID 0 // 舵机ID号

using namespace std;
using namespace fsuservo;

// 创建协议对象
FSUS_Protocol protocol(SERVO_PORT_NAME, FSUS_DEFAULT_BAUDRATE);
// 创建一个舵机对象
FSUS_Servo servo0(SERVO_ID, &protocol);

int main(){
    // 打印例程信息
```

```
cout << "Servo Data Read" << endl;
// 设置舵机角度(限制功率)
servo0.setAngle(0.0, 1000, 800);

while(true){
    // 信息查询
    uint16_t voltage = servo0.queryVoltage();
    uint16_t current = servo0.queryCurrent();
    uint16_t power = servo0.queryPower();
    uint16_t temperature = servo0.queryTemperature();

    // 打印状态信息
    cout << "voltage: " << voltage << endl;
    cout << "current: " << current << endl;
    cout << "power: " << power << endl;
    cout << "temperature: " << temperature << endl;

    // 延时1s
    protocol.delay_ms(1000);
}
}
```